

PGR Platform

Critical Gaps & Workarounds

Things that look and feel implemented — because a workaround makes them appear so — that must be made real before this platform can be treated as a live system for a real institution.

v1 · 2026-08-24 · reconciled against source + Non-Functional Criteria report

How to read this. Each item has four lines: what a user sees, what is really happening, why that will hurt in production, and what "real" means.

Severity	Meaning
CRITICAL	Will corrupt data, leak data, or lose money on go-live. Must be fixed before an institution touches it.
HIGH	Will not corrupt data, but will produce wrong operational outcomes or block go-live approval.
MEDIUM	Product debt: not blocking, but leaves the platform behaving less well than the UI suggests.

Executive summary

The PGR platform is far more real than most demo systems: 224 tests pass, the workflow engine really does write tasks and outbox events in the same transaction as the state change, and the settings page genuinely changes runtime behaviour. Where the surface still hides a workaround — and where those workarounds will hurt an institution on go-live — falls into six themes:

Nobody is actually connected. The integration hub is real and idempotent. But no partner system is sending anything: Finance never receives approved-stipend events, Research Office never sends award webhooks, HR never sends employee appointments. On go-live day, nothing crosses the wire.

Deployment does not exist. There is no Docker image, no CI, no backup/restore drill, no rate limiting, no TLS termination, no observability, no rollback runbook. The platform runs on a developer laptop today.

GDPR-critical endpoints missing. No subject-access export, no right-to-erasure endpoint, no documented retention schedule. For a system holding personal data on hundreds of researchers this is a legal blocker, not a nice-to-have.

Statutory returns are illustrative. The HESA profile is 12 fields chosen to prove the engine works, not a compliant return. Submitting the current output would be a governance incident.

Read-side infrastructure is single-node. Enterprise 360 is computed on every read; the replica session exists but no replica is provisioned; there is no cache and no materialised view. Fine at 266 students, fragile at 5000.

Optional modules are stubs where they should be flows. Person contacts/GDPR/merge, admissions conditional offers and fee-status/visa checks, completion classification and certificates, workflow SLA timers, richer portals, and reconciliation UI are all absent — the sidebar suggests parity that is not there.

The rest of this document lists the specific items, each with what a viewer sees, what is really happening under the hood, why it breaks things when a real institution tries to live on the platform, and what shipping the real version looks like.

Critical gaps

Fix before any institution touches production data.

[CRITICAL] 1. Outbound events emit into a void

What it looks like today. The Integration hub shows outbox events being dispatched, retries counted, dead-letters listed. The worker log confirms events go out. Everything looks operational.

What is actually happening. The events are HTTP-posted to `OUTBOX_TARGET_URL`. In dev this is unset or points at a scratch endpoint. **No partner system is subscribed.** Approved stipends do not reach Finance; graduation events do not reach the Records Office; funding-close events do not reach Research. The outbox is trying to deliver to nobody.

Why this breaks things at a real institution. On day one, Finance keeps paying stipends the platform believes are stopped, ceremony lists diverge from the platform's alumni list, and the platform's dashboards are correct while the surrounding systems are wrong. The audit trail will happily record every event delivered to *example.com*.

What making it real looks like. Contract with Finance (per `PGR_INTEGRATION_CONTRACT.md`) an endpoint per event type, exchange HMAC keys, run the 8-test go-live plan, and switch the target URL. Do the same for Research and HR. Until this is done, treat the platform as a system of record with no downstream.

[CRITICAL] 2. Inbound partner webhooks: nobody is sending

What it looks like today. The signed webhook endpoint is real: HMAC-verified, idempotent by `sourceId`, and known message types (research *award.created/updated*, HR *employee.appointed/updated*) actually mutate the database. Tests prove it.

What is actually happening. No partner system is **pointed at the webhook**. Research awards must be created manually in the app; HR-employee links must be created via the concurrent-identity endpoint by hand. The signed-webhook flow has never fired in anger. Finance messages have no handler — they land as *logged_only* and change nothing.

Why this breaks things at a real institution. Every award reference must be typed twice (once in the grants system, once here). Every dual-role person (PGR + staff member) must be linked manually — and if the administrator forgets, the platform will not detect it. Route B applicants can go through recruitment without their originating research demand ever being recorded because no one from Research Office had a system to send it from.

What making it real looks like. Get the URL registered on the partner side, share the HMAC secret, and turn on *employee.appointed* first (the highest-volume, safest event). Implement a Finance inbound handler for at least *payment.rejected* and *payment.confirmed* so the platform learns when a stipend it approved actually paid.

[CRITICAL] 3. TLS, rate limiting, and secret management: none

What it looks like today. The app runs locally over HTTP on 127.0.0.1. Everything works.

What is actually happening. There is no TLS termination in front of the API (S10 in the NFR report is ■ **not configured**). There is no rate limiting on the login endpoint or on any public route (S11 ■). `APP_SECRET_KEY` — which signs JWTs and inbound HMAC — is a value in a dev-checked-in `.env`.

Why this breaks things at a real institution. A public deployment can be brute-forced against the login endpoint at any rate; refresh-token theft over cleartext HTTP is trivial; anyone with the `.env` value can mint

valid tokens and forge inbound webhooks. The 5-strikes lockout is worthless if the attacker rotates through 10,000 valid usernames.

What making it real looks like. Deploy behind an HTTPS terminator (nginx / cloudfront / Traefik). Add per-IP and per-account rate limits on `/auth/*` and `/integration/inbound/*`. Move `APP_SECRET_KEY` to a real secret store; rotate on deploy.

[CRITICAL] 4. No backup, no restore drill, no rollback runbook

What it looks like today. PostgreSQL is running, the app is running, tests pass.

What is actually happening. There is no `pg_basebackup` schedule, no PITR archive, no tested restore procedure (D8 ■). There is no documented rollback path for a failed release (O9 ■). The Alembic downgrades are *generated* but never *run* against real data (R10 ■■).

Why this breaks things at a real institution. The first hardware fault or corrupted migration will lose everything. "How do we go back to Tuesday?" has no answer, and neither does "how do we undo migration 800191647194 after we saw the outage?"

What making it real looks like. Pick a Postgres backup service (Barman / pgBackRest / cloud-managed). Prove restore quarterly. Write the rollback runbook and test at least one downgrade end-to-end on a copy of production.

[CRITICAL] 5. GDPR-critical endpoints do not exist

What it looks like today. The platform holds personal data on every researcher, applicant and staff member. There is an audit trail. Users can be deactivated.

What is actually happening. **No subject-access export** (D6 ■) — asked for "give me everything you hold on me" the administrator has to write ad-hoc SQL across seven tables. **No erasure/anonymisation endpoint** (D7 ■) — the deactivate action leaves the row and all history intact. **No documented retention schedule** (D5 ■). Person contacts / GDPR / merge in Track 4B is **not started**.

Why this breaks things at a real institution. A subject-access request has a 30-day statutory clock in the UK. An institution cannot legally operate a system of record that cannot produce the full personal dataset or delete it on request. This is a blocker for information-governance sign-off, not a nice-to-have.

What making it real looks like. Build `GET /persons/{id}/export` that walks every table with a `person_id` FK and returns a portable file. Build `POST /persons/{id}/erase` that pseudonymises the person and hashes identifiers on audit/history rows (retention rules should not remove financial audit). Get the retention policy written by an information-governance owner and enforce it as a scheduled worker job.

[CRITICAL] 6. Statutory profile ships as an illustration, not a return

What it looks like today. Statutory returns page shows a HESA profile with 12 fields. Validate runs. Generate produces a CSV.

What is actually happening. The 12 fields were chosen to prove the mapping engine, not to satisfy the return. Coding frames, mandatory fields and validation rules for a real HESA return have **not been reviewed by anyone with HESA subject-matter authority**. The impl doc says exactly this in §13.

Why this breaks things at a real institution. Submitting the current CSV is a governance incident: incorrect student characteristics returned to the funding body invite corrections, questions and — occasionally — retrospective funding adjustments. An administrator who trusts the button labelled *Generate* will not know they submitted the wrong thing.

What making it real looks like. Sit a HESA specialist in front of the mappings for a real academic year. Add the missing fields (usually 40-70 depending on the return). Add the mandatory-field validator. Have Registry sign off the profile before it is promoted to a production year.

High-severity gaps

Will not corrupt data, but will block go-live approval or drive wrong operational outcomes.

[HIGH] 7. Object storage is the local filesystem

What it looks like today. Documents upload from the student record. Downloads work. Row scoping is enforced. Nothing looks off.

What is actually happening. Backend is `STORAGE_BACKEND=local`. Uploads land in a folder on the API server's disk (`STORAGE_ROOT`). There is no S3, no MinIO, no versioning, no lifecycle policy, no anti-virus scan.

Why this breaks things at a real institution. A single API-server host per environment now holds the only copy of every thesis draft, viva recording, waiver letter and ID scan. If the machine dies, the documents die. If two API servers come up (which any real deployment eventually needs), the file is on one host and missing from the other.

What making it real looks like. Enable the S3 backend that is already coded behind the `ObjectStore` interface. Give buckets lifecycle rules that match the retention policy from gap #5. Wire an AV scanner (ClamAV or a cloud equivalent) on upload.

[HIGH] 8. Enterprise 360 is computed on every read

What it looks like today. Analytics loads in a fraction of a second. Read p95 is 6ms. Everything looks fast.

What is actually happening. Enterprise 360 is **computed on read** through the replica session. There is no materialised view, no cache, no scheduled refresh. It is fast today because the demo has 266 students.

Why this breaks things at a real institution. At 5,000 students with 5+ years of history each, the aggregates that currently return in 43 ms will not. The read replica is *coded* but not *provisioned* (P6 ■■■). Dashboards will start dragging on the primary and slowing writes.

What making it real looks like. Add materialised views for the 360 lenses; refresh on schedule via the worker. Actually provision a read replica in production and point `DATABASE_REPLICA_URL` at it. Add a small cache in front of the executive dashboard tiles.

[HIGH] 9. Access-token permission changes take up to 15 minutes

What it looks like today. Add a permission to a role, the user gets it. The UI says so.

What is actually happening. Access tokens embed roles and permissions at issue time. They live for 15 minutes. A permission granted (or **revoked**) reaches an already-logged-in user only after their next token refresh. This is documented in §6.1.

Why this breaks things at a real institution. Revoking a leaver's access is not instant. Removing a permission after an incident is not instant. Institutions that expect a role change to be enforced *now* will find it isn't.

What making it real looks like. Add a JWT-revocation list keyed by `jwt.i` that the auth middleware checks on hot revocations (do not do this on every request — flag only). Provide an explicit `POST /admin/users/{id}/force-signout`.

[HIGH] 10. Deployment does not exist as an artifact

What it looks like today. `start-all.bat` works on a Windows dev machine.

What is actually happening. There is no Dockerfile for the app that has ever been built and run (there is one in the repo — check whether it works). There is no compose file. There is no Kubernetes manifest. There is no CI pipeline that runs tests on push. Environment setup is a batch file and a doc paragraph (O6 ■■■).

Why this breaks things at a real institution. "Move it to staging" is a multi-day project rather than a scripted step. Environment drift starts on day one because there is no single source of truth for what runs in prod.

What making it real looks like. Ship a minimal Docker image for the API and the worker. A compose file for local integration. GitHub Actions running the test suite on every PR. A one-command `make deploy` against staging.

[HIGH] 11. Observability: nothing to look at when it breaks

What it looks like today. Structured logs include a request-id per line. Log lines look useful.

What is actually happening. There is no OpenTelemetry export, no Prometheus scrape, no Sentry (O7 ■, O8 ■ **config slot exists, not wired**). When someone reports "it was slow at 10am" the only tool is grepping the log file on the host.

Why this breaks things at a real institution. SLA reports cannot be produced without observability. Incidents will take longer to root-cause than they should. Distributed traces across the worker/outbox/webhook path are unavailable, which is where most integration bugs will hide.

What making it real looks like. Add OpenTelemetry SDK to the FastAPI app and the worker; export to whatever the institution uses (Datadog, Grafana Cloud, Honeycomb, whatever). Wire the Sentry DSN. Add golden-signal dashboards (latency, errors, saturation, traffic) for the API and the outbox dispatcher.

[HIGH] 12. Older students have broken funding lineage

What it looks like today. A student's funding card shows the current arrangement and past payments. Everything looks fine.

What is actually happening. Students created **before Phase 6** have no `research_project` row and therefore no link to their award. Their funding lineage shows integrity warnings, and the cohort funding-integrity screen highlights them as errors. There is **no backfill script**. §13 admits it.

Why this breaks things at a real institution. The *Funding integrity* screen — which is meant to signal *a real problem needs attention* — is polluted by rows that are only "broken" because the data is old. Real administrators will learn to ignore that screen, which defeats the point.

What making it real looks like. Write `scripts/backfill_project_links.py`: for each pre-Phase-6 student, either link an existing project or create a stub project row referencing the best-guess award. Log every uncertain case for manual review. Delete the workaround comment.

[HIGH] 13. Ask PGR is read-only and one Tier-3 policy item is unresolved

What it looks like today. The assistant answers natural-language questions accurately. Users assume they can also ask it to *do* things.

What is actually happening. By design, this release is read-only. Track 5.2 (writes with confirm) is not started. The Tier-3 blocked-action list has one item the customer selected as *Something else*, still unspecified. The assistant will always refuse writes today.

Why this breaks things at a real institution. Two consequences: (a) users lose trust — "why does it say *Read-only, I can't change anything* when it obviously understood?" (b) an eventual 5.2 release without the Tier-3 policy nailed down risks the assistant doing something the institution never sanctioned.

What making it real looks like. Get the Tier-3 policy signed off by the CIO. Ship 5.2 with a confirm-before-write pattern already prototyped in `docs/PGR_ASSISTANT_DESIGN.md`. Until then, adjust the UI copy to say *I only **find** things — for changes, open the record.*

Medium-severity gaps

Product debt: not blocking, but the UI hides that the flow underneath is thin.

[MEDIUM] 14. Person: contacts / GDPR / merge missing

What it looks like today. Person 360 shows identities and a timeline. It looks complete.

What is actually happening. Contact records (phone, next-of-kin, right-to-contact flags) do not exist. Person **merge** — the operation you need when two records for the same human end up in the database (very common on any real go-live) — is not implemented. Track 4B.1 *not started*.

Why this breaks things at a real institution. Duplicate records will accumulate silently after any bulk import from Registry or the applicant portal, and the deterministic HR matcher will produce *ambiguous* → *task* outcomes that no one has a UI to resolve. Neither GDPR retention nor "do not contact" preferences can be recorded.

What making it real looks like. Build person merge (choose surviving row, rewrite FK references, close the duplicate). Add contacts with typed channels and a `do_not_contact` flag that the notification sender respects.

[MEDIUM] 15. Recruitment: no references, no interviews as first-class objects

What it looks like today. The application detail shows a stage history and assessments. It reads as complete.

What is actually happening. Reference requests and interview scheduling do not exist as domain objects. Any real institution wants: nominate two referees, send them a token-linked reference form, record the response; schedule interview panels, notify participants, record the outcome. Track 4B *not started*.

Why this breaks things at a real institution. Administrators fall back to email plus a Word document, then paste the outcome into an assessment. There is no audit trail of the reference request or the interview schedule.

What making it real looks like. Add `reference_request` with token-linked external submission. Add interview with panellists, calendar events (via existing outbox), and outcome capture.

[MEDIUM] 16. Admissions: no conditional offers, no fee-status/visa gating

What it looks like today. Offers move draft → issued → accepted. Simple and clean.

What is actually happening. There is no *conditional* offer type ("accept subject to passing your Masters at 2:1+") and no fee-status/visa capture that would gate international students. Track 4B *not started*.

Why this breaks things at a real institution. Real admissions almost always use conditional offers. Without them, administrators cannot record the condition, cannot track whether the condition was met, and have no built-in way to withdraw an unfulfilled conditional offer. Fee-status errors on international students are directly linked to compliance risk with UKVI.

What making it real looks like. Add `OfferCondition` (many per offer) with a satisfy-by date and evidence reference. Add fee-status + visa fields on the applicant record with a required-before-issue check.

[MEDIUM] 17. Completion: classification and certificates not modelled

What it looks like today. The completion screen has a **Graduate** button. Graduation runs.

What is actually happening. Award *classification* (Pass, Merit, Distinction, PhD-with-corrections vs. PhD-with-none) is a free field, not an enum with a workflow. Certificate generation (the PDF diploma) is not implemented. Track 4B *not started*.

Why this breaks things at a real institution. The Records Office cannot rely on the platform for the actual deliverable of graduation — the certificate — and has to run a parallel process in a Word template, which is where classification errors get introduced.

What making it real looks like. Add a governed classification workflow (proposed → confirmed by exam board → published). Ship a certificate PDF template with the platform's typography; generate on the graduation transaction.

[MEDIUM] 18. Workflow SLA timers are absent

What it looks like today. Overdue rows exist (meetings, tasks). The worker escalates on its 60-second interval.

What is actually happening. There is no first-class notion of an **SLA clock** on a task or workflow step. A task is either in-progress or overdue against a fixed deadline; you cannot see "we have promised a 5-working-day turnaround on assessments and we are running at 6 for the last month."

Why this breaks things at a real institution. Institutional service levels (turnaround on offer decisions, corrections sign-off, viva scheduling) cannot be reported on. Any SLA meeting has to be reconstructed from the audit log by hand.

What making it real looks like. Add TaskSLa with target duration, computed elapsed, working-day-aware clock. Surface an SLA dashboard alongside Analytics.

[MEDIUM] 19. Integration reconciliation UI is thin

What it looks like today. GET `/integration/reconciliation` exists and the page renders three sections (stuck outbound, failed inbound, waiting on a person).

What is actually happening. The reconciliation UI covers detection but not **remediation**. There is no bulk retry, no "resolve waiting-on-a-person by assigning the person here", no export. Track 4B *not started*.

Why this breaks things at a real institution. When integration errors accumulate (see gap #1) the administrator has no tool to work through them in bulk. Individual dead-letter replays are one-at-a-time.

What making it real looks like. Add bulk selection + replay to the dead-letter section. Add "assign person" inline in the waiting section. Add CSV export for post-incident review.

[MEDIUM] 20. Richer portals still missing

What it looks like today. The student portal renders *My journey* cleanly. The supervisor portal renders the caseload.

What is actually happening. Both are essentially read-only summaries with a few actions. Missing: student ability to accept a meeting time from a supervisor-proposed slot, to submit corrections through the portal, to see and acknowledge conditions on their progression outcome. Supervisor: capacity request, sabbatical

declaration, delegation to a co-supervisor.

Why this breaks things at a real institution. Users bypass the portal and email the office — which puts the operational load right back on administrators the portal was supposed to relieve.

What making it real looks like. Prioritise the two flows that actually move volume: (a) student uploads correction evidence + admin sign-off; (b) supervisor proposes a meeting → student accepts → meeting logged automatically.

[MEDIUM] 21. Reporting: cohort and trend analysis missing

What it looks like today. The reporting tab shows Enterprise 360 with five lenses. Feels rich.

What is actually happening. The lenses answer "what is the state today?" There is no **cohort trend** ("how did the 2023 intake progress compared to 2022?") and no **forecast band** around the completion prediction. Track 4B *not started*.

Why this breaks things at a real institution. Executives asked for "how are we doing" get an answer for today only. Board reports need year-over-year cohort comparisons and end up being built manually in Excel — which was one of the outcomes this platform was commissioned to remove.

What making it real looks like. Add cohort-level reports parameterised by intake year. Add confidence bands on the completion forecast. Export both to CSV via the existing exports module.

[MEDIUM] 22. Recruitment pipeline UI: no route badges/filters

What it looks like today. Route information exists on every application and reaches the export and Enterprise 360.

What is actually happening. The recruitment list does not display the route badge and does not offer a route filter. FE-6.2 is admitted as *not started* in §13.

Why this breaks things at a real institution. Administrators cannot triage "show me all Route B applications this week" from the pipeline without opening each application. Small item, but exactly the sort of thing that erodes trust in a list view.

What making it real looks like. Add the route badge column and the filter chip. One frontend PR.

[MEDIUM] 23. Emails outside the app are opinion-free

What it looks like today. Notifications land in the notification centre and are emailed to users who opted in.

What is actually happening. Templates are minimal and unbranded. There is no **preferences quiet-hours**, no digest option, no unsubscribe link, no bounce-tracking. Email delivery is a template plus a fire-and-forget SMTP send.

Why this breaks things at a real institution. Notification fatigue is high (every task raises an email); leavers keep receiving mail from a bounce the app cannot see; institutional email hygiene rules (opt-out language, sender identity, list-unsubscribe) are unmet.

What making it real looks like. Add branded HTML templates. Add quiet hours and daily-digest preference on top of the existing notification-preference model. Wire a bounce handler (via the SMTP provider) that deactivates the email channel for a hard bounce.

Policy debt (not built-vs-not-built, but agreed-vs-not-agreed)

Several defaults are shipped as institutional decisions but were **chosen by the engineering team, not signed off by the customer**. Any real institution should ratify these before go-live. The values are wired via the settings registry (Phase 8), so changing them is one PUT — but changing them at 3am after an incident is not how policy should be set.

Setting	Default	Question the institution needs to answer
supervision.max_supervisees	8	What is our real capacity ceiling per supervisor?
supervision.expected_meeting_interval_days	90	How often should supervision meet? (Full-time vs part-time)
lifecycle.part_time_factor	2.0	Do we treat part-time as exactly 2× full-time, or 1.5× (typical)?
funding.min_gap_days	7	How large a gap between funding arrangements is a problem worth flagging?
corrections.minor_days	28	Is minor-correction turnaround 28 days as we assumed?
corrections.major_days	182	Is major-correction turnaround 6 months?
auth.MAX_FAILED_LOGINS / LOCKOUT_MINUTES	5 / 15	Does IT security accept this lockout curve?
progression.appeal_window_days	28	Legally, what is the appeal window in our regulations?
reports.retention_years	unset	How long do we retain per data class? (feeds gap #5)
assistant.llm_enabled	false	Do we ever want the LLM path on? Under what policy?

Suggested order of attack

If the goal is "an institution can start using this in the next six weeks," the sequence that buys the most safety per week is:

- Week 1 — Deployment artefact (gap #10), TLS + rate limiting (#3), backup + restore drill (#4).
- Week 2 — GDPR endpoints and retention policy (#5), documented rollback runbook (#4).
- Week 3 — Integration go-live for HR appointments and Finance stipend confirmations (#1, #2), reconciliation UI (#19).
- Week 4 — HESA statutory profile signed off with Registry (#6), backfill project links (#12).
- Week 5 — Observability wired (#11), object storage on S3 (#7), materialised 360 views (#8).
- Week 6 — Person merge + contacts (#14), workflow SLA timers (#18), route badges (#22).

Everything on this list is either an admission from the impl doc's *Deliberately not built yet* section, an unmet criterion from the Non-Functional Criteria report, or a documented workaround inside the code. Nothing here is speculative.